

LLM PROMPTING – RAG

© DAN MITCHELL

Outline

- Introduction to RAG
- Semantic Similarity & Database search
- Building a RAG

RAG

- One downside of generative LLMs is that specific domain knowledge is tricky
 - LLMs learn sentence composition rules
 - LLMs don't learn what part is needed to fix a tractor
- **Retrieval Augmented Generation (RAG)** is one common solution

RAG

- We can search a tractor repair manual for information relevant to a question!
 - Retrieval
- We can incorporate the relevant parts (**Augmentation**) of the manual in an LLM prompt
 - Generation

RAG

1. Ask a domain specific question
2. Search a database of domain specific information
3. Find the most relevant parts of the database
4. Design a prompt to an LLM that has the question and results from step 3

RAG

Answer the following question with the context below.

Question: Where is Dan from?

Context:

Dan is a professor at UT.

Dan grew up in Houston.

Dan lives in Austin now.

Answer:

Dan is from Houston ✓

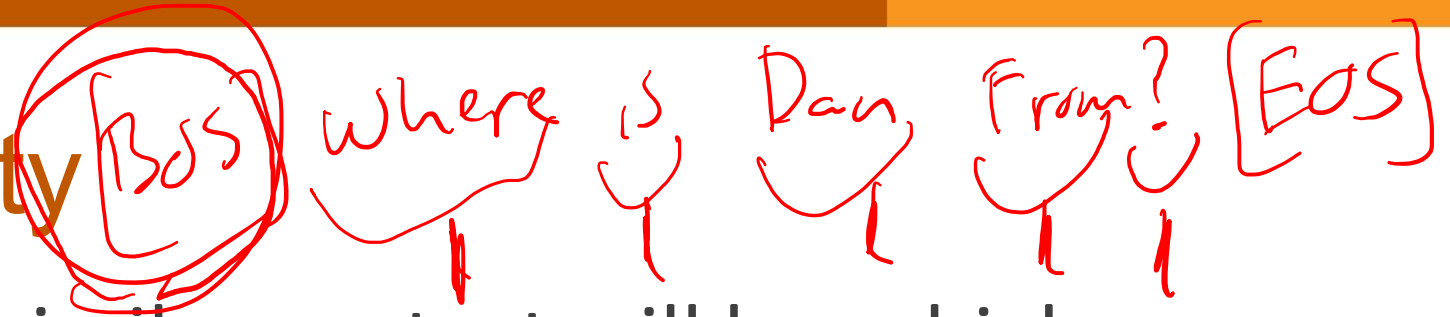
RAG

- How do we find the best context to put into our prompt?
- Semantic similarity
 - The weather is nice outside
 - It's 75 degrees and partly cloudy today!
 - We are outside today partly because 75 nice clouded leopards are a few degrees north of here.
- We can get an embedding for each token in each block using an encoder transformer
 - How do we use those to determine similarity?

Semantic Similarity

- Start with a BERT model
- Feed text-block-A and text-block-B into BERT
- Get contextually aware embeddings for each token in each block
- Average the embeddings in text-block-A
- Average the embeddings in text-block-B
- Calculate cosine similarity of 2 averages

Semantic Similarity



- Sentences that have similar context will have high similarity
- This isn't perfect though
- Fine tune BERT
 - Dataset of similar blocks of text
 - Overflow repeat tagged questions
 - Dataset of question/answer pairs
 - ELI5
- [BOS] token
- There are many pre-trained models available

Vector Database

- Break your tractor manual into lots of chunks
 - Most models only accept 512 tokens...
- Embed every chunk using a pretrained model
 - Store each embedding in a database
- FAISS / Pinecone / Chroma / Elastic / ...

Semantic Search

- A question comes in
- Embed it using the pretrained model
- Calculate similarity between question and each entry in the database
 - This is FAST!
- Grab the top few hits
 - Rank cutoff, threshold cutoff, ...

Augment Prompt

- Design your LLM prompt to put question and results together to answer question!

Answer the following question with the context below.
Question: Where is Dan from? → EM → (11) — (11) (2) (1) (3)
Context:
Dan is a professor at UT.
Dan grew up in Houston.
Dan lives in Austin now.
Answer: Dan is from Houston.

RAG Software

- LangChain / LlamaIndex

- C3 AI

- Westlaw Precision

Thompson

LangChain

1. Create database
 - a. Input directory of all documents
 - b. Split each document into overlapping chunks
 - c. Embed each chunk
 - d. Save to database (FAISS)
2. Embed query
3. Find top results
4. Feed results and query to generative LLM

RAG Evaluation

- How do we know if the RAG's answer is good or not?
 - Errors like hallucination or irrelevant results can occur
- We need a way to systematically analyze our RAG
- Evaluation improves the accuracy and relevance of our output

RAG Evaluation

- **Groundedness** - Is the answer clearly based on the retrieved documents?
- **Relevance** - Does the answer directly respond to the user's question?
- Is there better context in the database?

RAG Evaluation

- How do we evaluate?
 - **ROUGE/BLEU**: Compare output to a reference (gold-standard) answer
 - **LLM-as-Judge**: Show the answer to an LLM and ask it to evaluate
- Good practice for the evaluator to be a different LLM than the generator

Agentic RAG

- What if we have multiple databases to search?
- What if we have multiple search methodologies?
- Let an LLM decide!
 - Agentic RAG